

Resolução dos Exercícios

Capítulo: Ponteiros

1. Suponha o seguinte esquema que corresponde à declaração que se segue:

```
char s[] = "Maria", *ptr = s;
```

M	a	r	i	a	\0				1000		
1000	1001	1002	1003	1004	1005	...	...	...	5000	5001	...

Complete a próxima tabela, colocando as expressões que dão como resultado o conteúdo da coluna Valor

Usando s	Usando ptr	Valor
s[2]	*(ptr+2)	'r'
&s[0]	ptr	1000
&s[1]	ptr+1	1001
s + ((char *)&ptr - s)	&ptr	5000

2. Responda, sucintamente, às seguintes questões:

2.1 Qual operador que nos permite obter o endereço de uma variável?

&

2.2 Qual o caractere que se coloca na declaração de uma variável para indicar que é um ponteiro?

*

2.3 Onde se coloca esse caractere?

Entre o identificador e o tipo da variável

2.4 O que contém uma variável do tipo ponteiro?

Um endereço de memória

2.5 Qual o símbolo que podemos colocar em um ponteiro para indicar que ele não aponta para nada?

NULL

2.6 Como se designa e qual o aspecto do operador que permite saber o que se encontra na posição de memória armazenada em um ponteiro?

*ptr - O * indica o conteúdo do endereço guardado em ptr

2.7 Se ptr for um ponteiro, qual o valor de *(&ptr)

O valor será o próprio ptr, ou seja, o endereço da variável para a qual ptr aponta

2.8 Indique duas formas de se obter o elemento inicial de um vetor v.

v[0], *v

2.9 Por que razão podemos utilizar ponteiros para percorrer os vetores existentes em memória

Porque o nome de um vetor funciona como um ponteiro para o seu elemento inicial e, como os elementos de um vetor são armazenados sequencialmente na memória, podemos acessar qualquer

posição usando aritmética de ponteiros.

2.10 Como declaramos uma variável que tenha a capacidade de conter o endereço de um ponteiro para float?

```
float **ptr;
```

3. Implemente a seguinte função

```
char *strrchr(char *s, char ch)
```

que retorna o endereço da última ocorrência de ch em s; caso não exista retorna NULL

Escreva ainda um programa que solicite um nome completo e escreva na tela apenas o sobrenome

[Ver código-fonte \(exe03.c\)](#)

4. Implemente a seguinte função

```
char *strstr(char *str1, char *str2)
```

retorna o endereço de str1 em que ocorre pela primeira vez a substring str2. Caso não exista, retorna NULL

[Ver código-fonte \(exe04.c\)](#)

5. Exercício de Exame

Implemente a seguinte função

```
char *first_vogal(char *s)
```

retorna o endereço em que ocorre a primeira vogal na string s. Caso não exista, retorna NULL

[Ver código-fonte \(exe05.c\)](#)

6. Exercício de Exame

Implemente a seguinte função

```
char *strins(char *dest, char *orig)
```

insere a string orig no início da string dest, retornando dest

ex:

```
char s[100] = "Autônoma";
```

```
strins(s, "Universidade");
```

```
printf(s) --> UniversidadeAutônoma
```

[Ver código-fonte \(exe06.c\)](#)

7. Exercício de exame.

Suponha a seguinte declaração

```
int v[5] = {10, 20, 30, 40, 50}; /* vetor com 5 inteiros */
```

```
int *pv; /* ponteiro para inteiro */
```

```
int **ppv; /* ponteiro para ponteiro para inteiro */
```

a) Qual o conjunto de instruções que você escreveria, de tal modo que as três variáveis ficassem, de alguma forma, relacionadas entre si?

```
pv = v; ppv = &pv;
```

b) Represente, em um esquema de memória, qual o estado em que ficou a memória depois de executadas as instruções do item (a), supondo que a dimensão de todos os tipos envolvidos é de um byte. Dê os valores que ache apropriados aos endereços de todas as variáveis envolvidas

10	20	30	40	50		0x7fff5e8f4ee3
0x7fff5e8f4ee3	0x7fff5e8f4ee4	0x7fff5e8f4ee5	0x7fff5e8f4ee6	0x7fff5e8f4ee7	...	0x7fff5e8f4ed8

c) Quais os parâmetros que se deve colocar em cada printf de tal forma que se obtenha a seguinte saída, sendo o primeiro parâmetro definido com base em v, o segundo com base em pv e o terceiro com base em ppv?

```
printf("%d %d %d\n", , , ); --> 10 10 10
printf("%d %d %d\n", , , ); --> 20 20 20
printf("%d %d %d\n", , , ); --> 30 40 50
printf("%d %d %d\n", , , ); --> 9 19 29
```

```
printf("%d %d %d\n", *v, *pv, **ppv);

printf("%d %d %d\n", *(v+1), *(pv+1), *((*ppv)+1));

printf("%d %d %d\n", *(v+2), *(pv+3), *((*ppv)+4));

printf("%d %d %d\n", (*v) - 1, *(pv + 1) - 1, *((*ppv)+2) - 1);
```

d) Qual a saída da seguinte instrução?

```
printf("\n%d %d %d", &v[0], &pv, &ppv);
```

&v[0] -> Endereço do primeiro elemento do vetor

&pv -> Endereço do ponteiro pv

&ppv -> Endereço do ponteiro para ponteiro ppv

Como os especificadores usados em printf são todos %d, os respectivos endereços serão mostrados em formato de um número inteiro.

8. Exercício de Exame

Implemente a seguinte função

`char *strDelUpper(char *s)`
apaga todos os caracteres maiúsculos em s

[Ver código-fonte \(exe08.c\)](#)

9. Exercício de Exame

Implemente a seguinte função

`char *strDelStr(char *s1, char *s2)`

apaga em s1 a primeira ocorrência de s2.

`char *s = "O rato roeu a rolha da garrafa de rum do rei da Rússia"`

`strDelStr(s, "xy") --> "O rato roeu a rolha da garrafa de rum do rei da Rússia"`

`strDelStr(s, "ra") --> "O to roeu a rolha da garrafa de rum do rei da Rússia"`

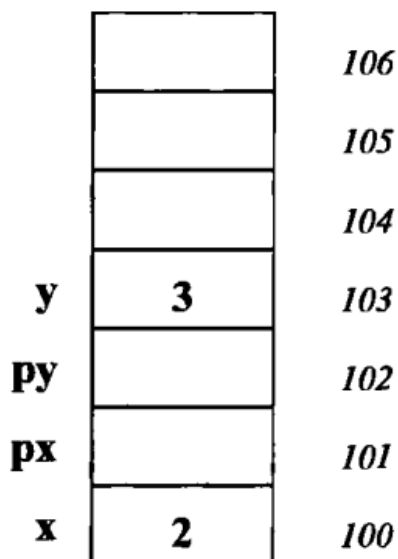
[Ver código-fonte \(exe09.c\)](#)

10. Exercício de exame

Suponha a seguinte declaração:

```
int x = 2, *px, *py, y = 3;
```

que corresponde ao seguinte esquema de memória:



Suponha que a escrita de inteiros e ponteiros pode ser feita através da função `printf` usando o formato `%d`

a) Escreva o código necessário para colocar `px` apontando para `x` e `py` apontando para `y`.

```
px = &x; py = &y;
```

b) Depois de executado o item (a), qual a saída das seguintes instruções?

```
printf("%d %d\n", x, y);  
printf("%d %d\n", *px, *py);  
printf("%d %d\n", &px, &py);
```

`printf("%d %d\n", x, y);` - Imprime o valor de x e y, 2 e 3 respectivamente

`printf("%d %d\n", *px, *py);` - Imprime o valor da variável que é apontada por px e py; px aponta para x e py aponta para y, logo será mostrado 2 e 3.

`printf("%d %d\n", &px, &py);` vai mostrar o endereço do ponteiro px e do ponteiro py.

c) Caso seja feito `px = py`, qual a saída de

```
printf("%d %d %d %d %d %d %d %d", x, &x, px, *px, y, &y, py, *py)
```

Considerando

```
int x = 2, *px, *py, y = 3;
```

```
px = &x;
```

```
py = &y;
```

```
px = py;
```

Na ordem será mostrado:

```
0 valor da variável x --> 2
0 endereço da variável x
0 endereço do ponteiro px
0 conteúdo da variável apontada por px que é y --> 3
0 valor da variável y --> 3
0 endereço da variável y
0 endereço do ponteiro py
0 conteúdo da variável apontada por py que é y --> 3
```